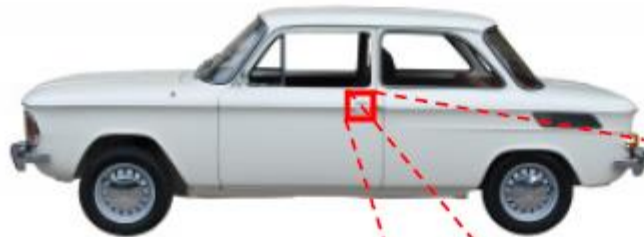


# Artificial Neural Networks

# What is this?

You see this:



But the camera sees this:

|     |     |     |     |     |     |     |     |     |     |     |     |
|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|
| 194 | 210 | 201 | 212 | 199 | 213 | 215 | 195 | 178 | 158 | 182 | 209 |
| 180 | 189 | 190 | 221 | 209 | 205 | 191 | 167 | 147 | 115 | 129 | 163 |
| 114 | 126 | 140 | 188 | 176 | 165 | 152 | 140 | 170 | 106 | 78  | 88  |
| 87  | 103 | 115 | 154 | 143 | 142 | 149 | 153 | 173 | 101 | 57  | 57  |
| 102 | 112 | 106 | 131 | 122 | 138 | 152 | 147 | 128 | 84  | 58  | 66  |
| 94  | 95  | 79  | 104 | 105 | 124 | 129 | 113 | 107 | 87  | 69  | 67  |
| 68  | 71  | 69  | 98  | 89  | 92  | 98  | 95  | 89  | 88  | 76  | 67  |
| 41  | 56  | 68  | 99  | 63  | 45  | 60  | 82  | 58  | 76  | 75  | 65  |
| 20  | 43  | 69  | 75  | 56  | 41  | 51  | 73  | 55  | 70  | 63  | 44  |
| 50  | 50  | 57  | 69  | 75  | 75  | 73  | 74  | 53  | 68  | 59  | 37  |
| 72  | 59  | 53  | 66  | 84  | 92  | 84  | 74  | 57  | 72  | 63  | 42  |
| 67  | 61  | 58  | 65  | 75  | 78  | 76  | 73  | 59  | 75  | 69  | 50  |

# Computer Vision: Car detection



Cars



Not a car

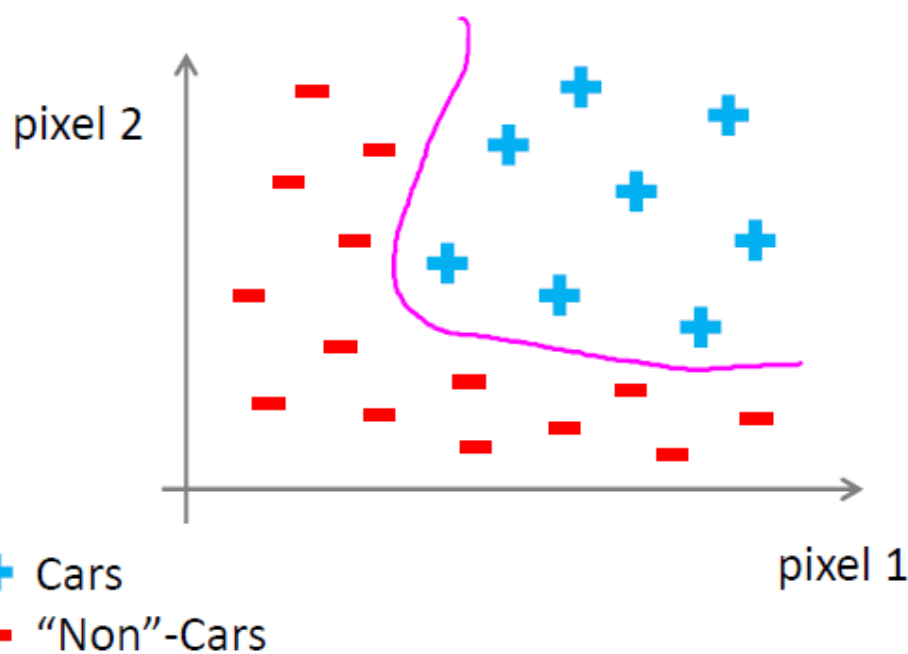
Testing:



What is this?



Learning  
Algorithm



50 x 50 pixel images  $\rightarrow$  2500 pixels

$n = 2500$  (7500 if RGB)

$$\rightarrow x = \begin{bmatrix} \text{pixel 1 intensity} \\ \text{pixel 2 intensity} \\ \vdots \\ \text{pixel 2500 intensity} \end{bmatrix}$$

0-255

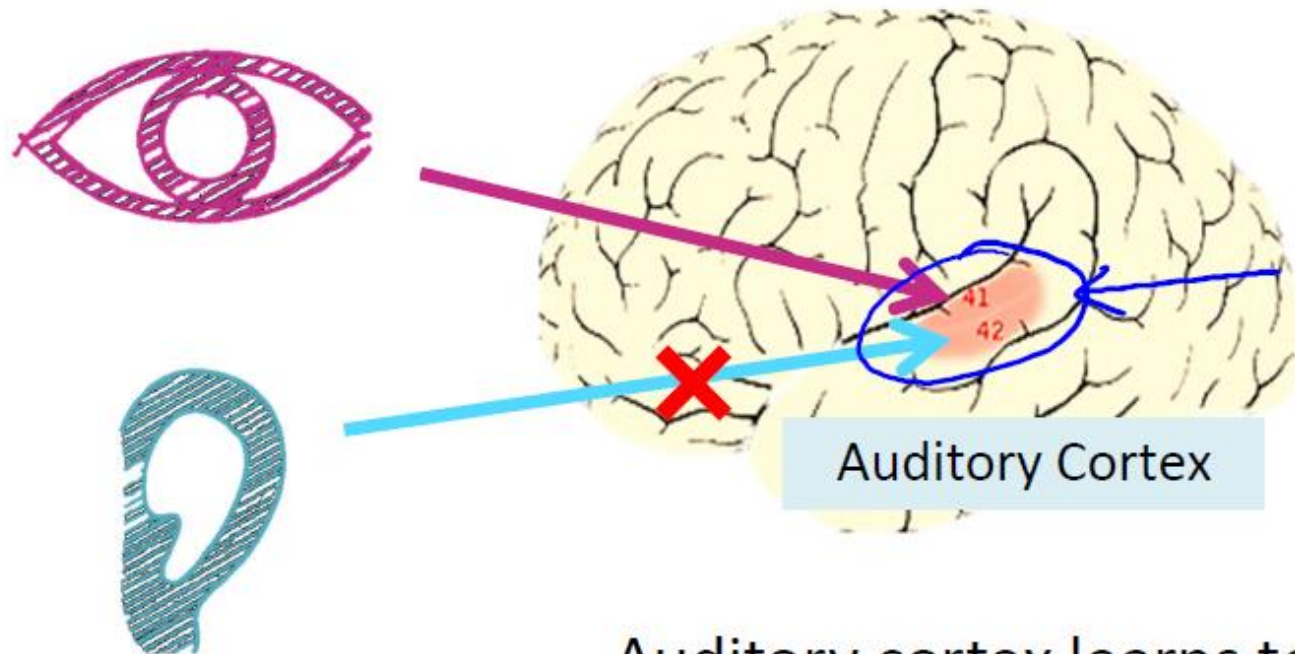
Quadratic features ( $x_i \times x_j$ ):  $\approx$  3 million features

## Neural Networks

- Origins: Algorithms that try to mimic the brain.
- Was very widely used in 80s and early 90s; popularity diminished in late 90s.
- Recent resurgence: State-of-the-art technique for many applications

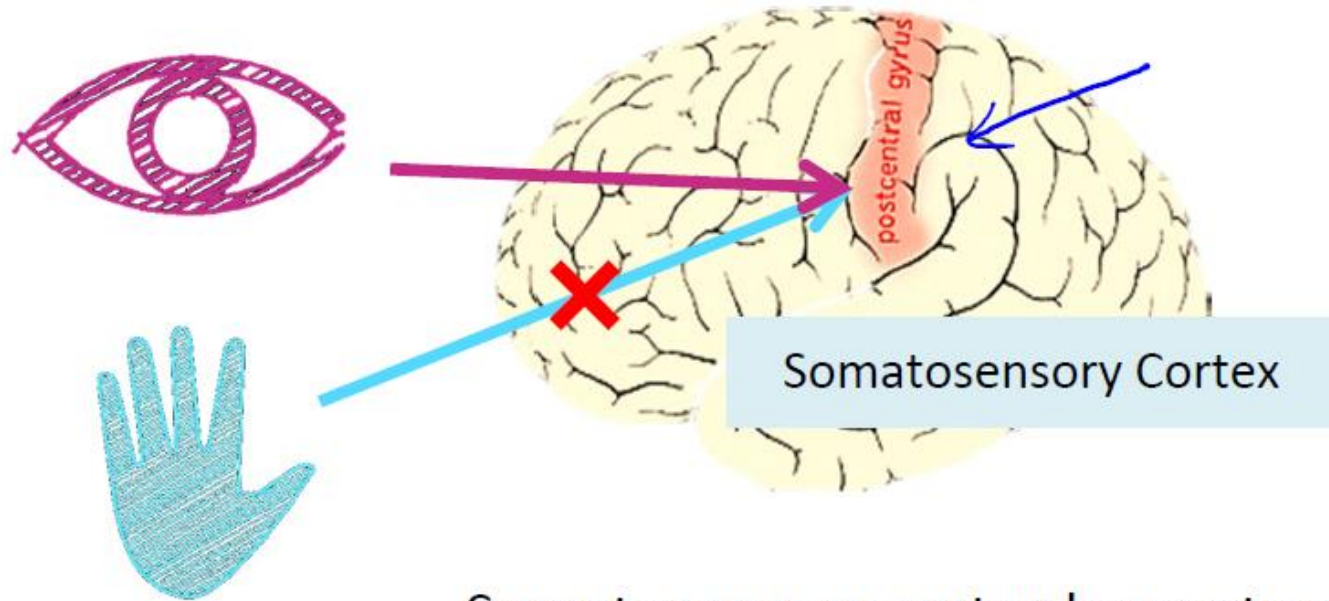


# The “one learning algorithm” hypothesis



Auditory cortex learns to see

## The “one learning algorithm” hypothesis



Somatosensory cortex learns to see

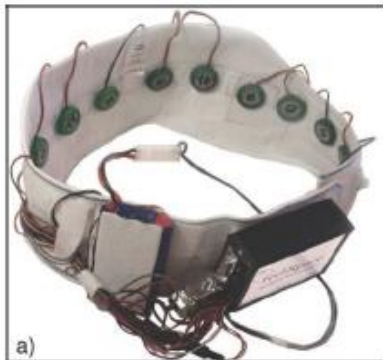
# Sensor representations in the brain



Seeing with your tongue



Human echolocation (sonar)



Haptic belt: Direction sense



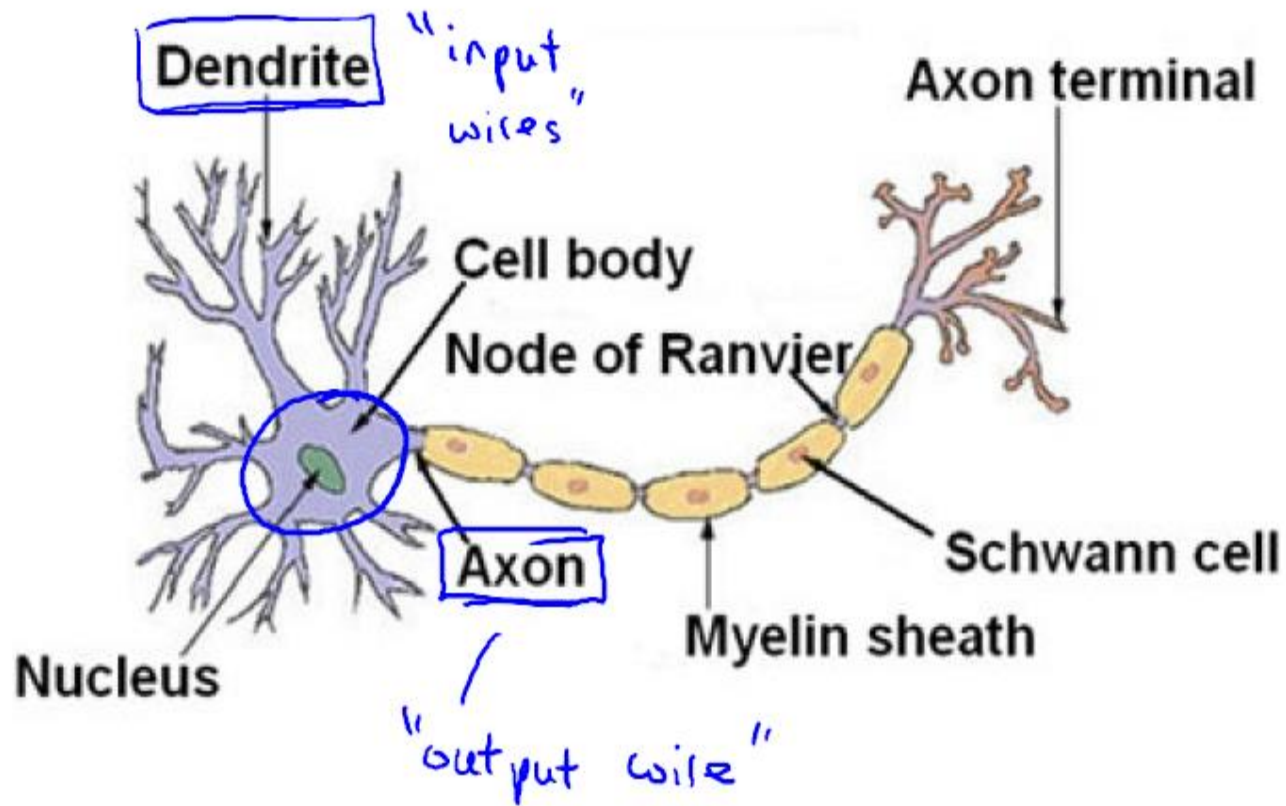
Implanting a 3<sup>rd</sup> eye

[BrainPort; Welsh & Blasch, 1997; Nagel et al., 2005; Constantine-Paton & Law, 2009]

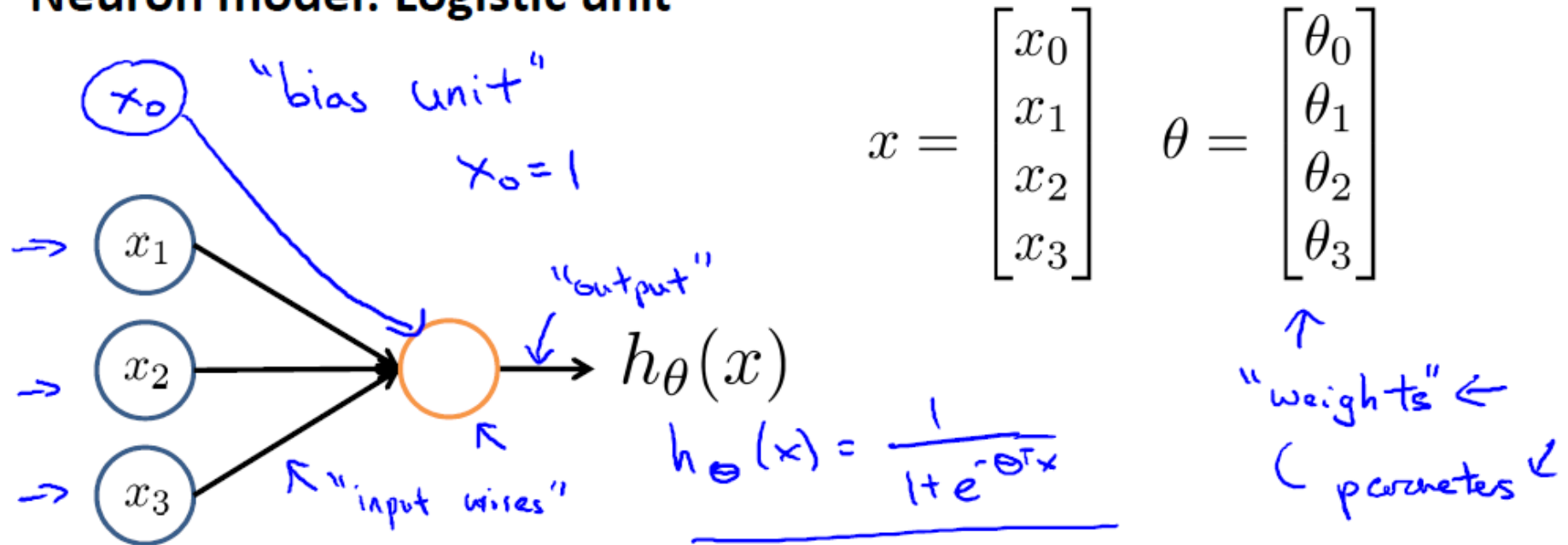
Andrew Ng



## Neuron in the brain



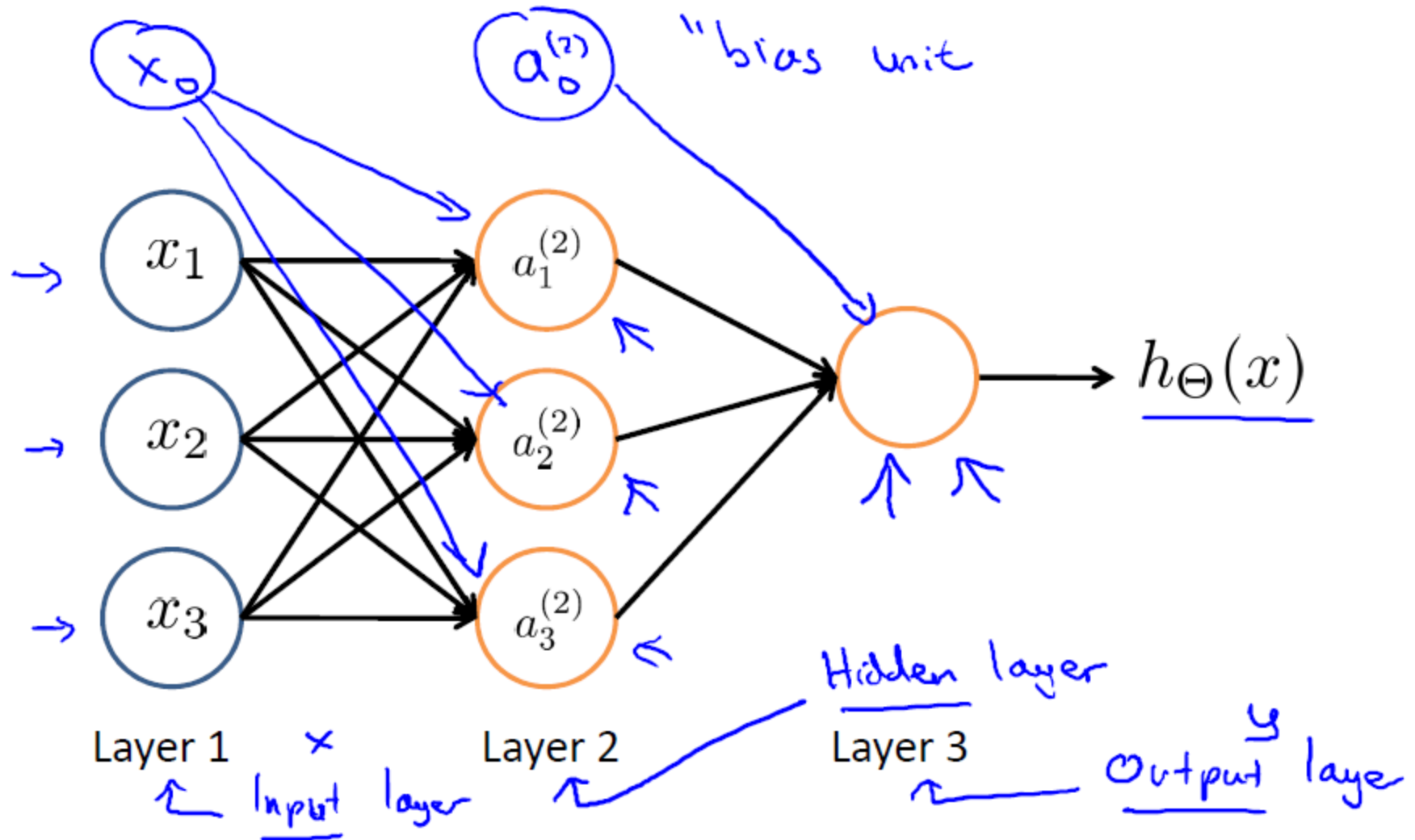
## Neuron model: Logistic unit



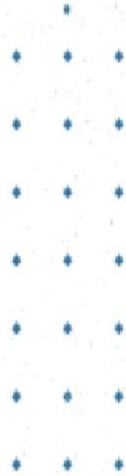
Sigmoid (logistic) activation function.

$$g(z) = \frac{1}{1 + e^{-z}}$$

# Neural Network

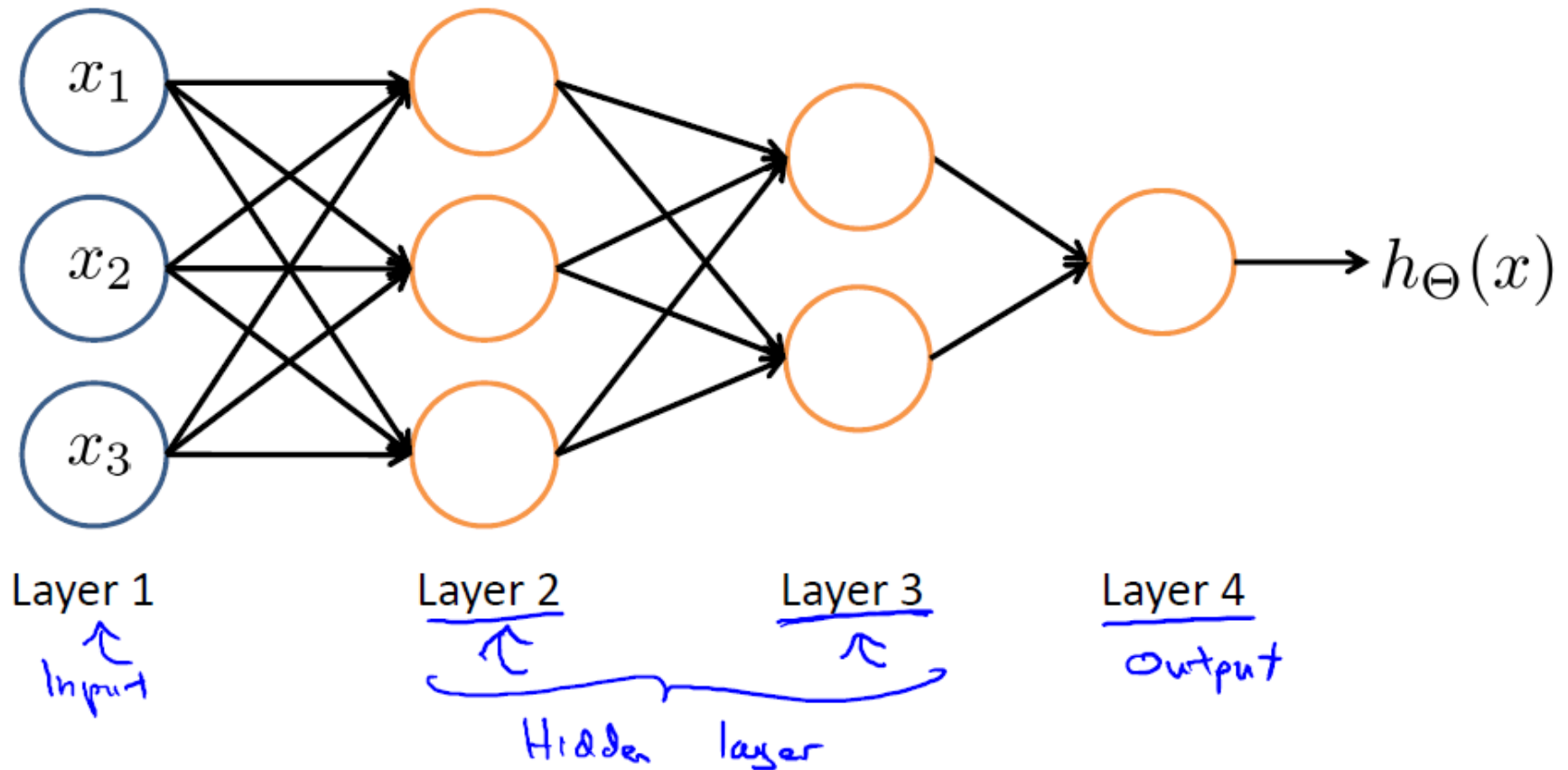


# TO DO: Forward Propagation





## Other network architectures



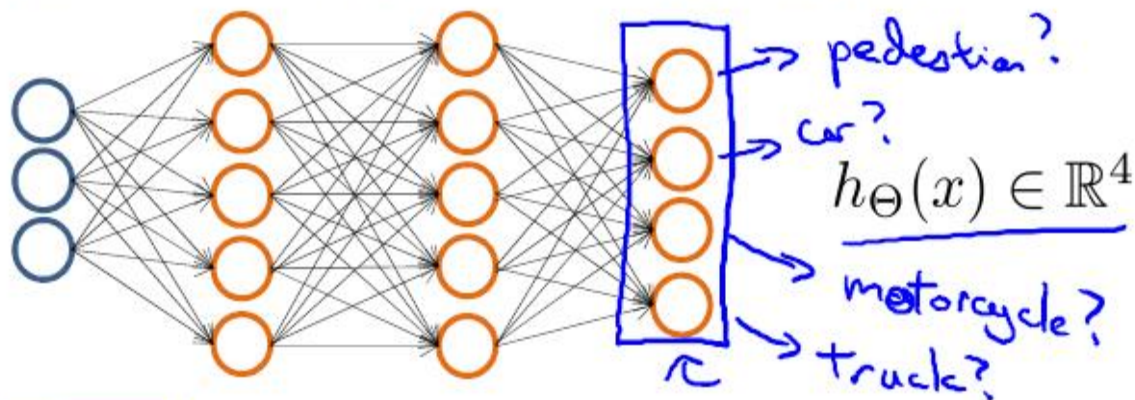
Andr

# We Don't want to buy from an unreliable Merchant

$$x_p \cdot w_1 + x_b \cdot w_2 + x_r \cdot w_3 = y$$

- If  $y \geq t \rightarrow \text{buy}(1)$
- If  $y < t \rightarrow \text{Do not buy}(0)$
- $w_1?$
- $w_2?$
- $w_3 \rightarrow$

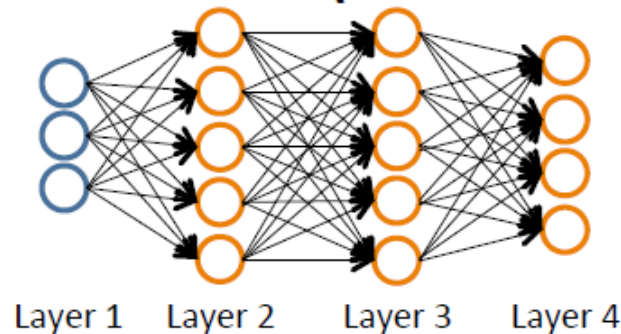
## Multiple output units: One-vs-all.



Want  $h_{\Theta}(x) \approx \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}$ ,  $h_{\Theta}(x) \approx \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}$ ,  $h_{\Theta}(x) \approx \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \end{bmatrix}$ , etc.

when pedestrian                      when car                      when motorcycle

## Neural Network (Classification)



### Binary classification

$y = 0$  or  $1$

1 output unit

$\{(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \dots, (x^{(m)}, y^{(m)})\}$

$L =$  total no. of layers in network

$s_l =$  no. of units (not counting bias unit) in layer  $l$

### Multi-class classification (K classes)

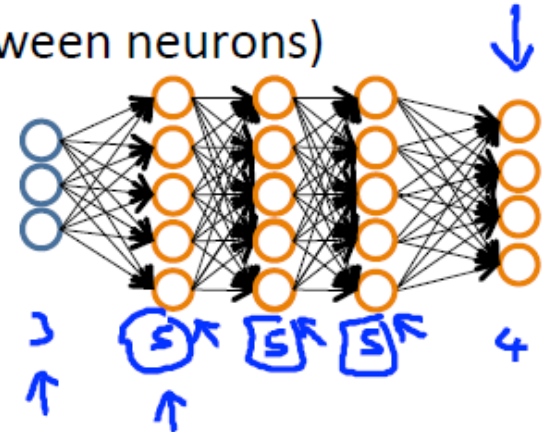
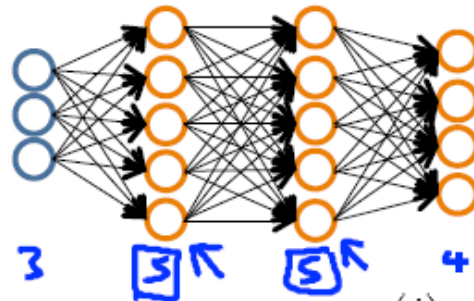
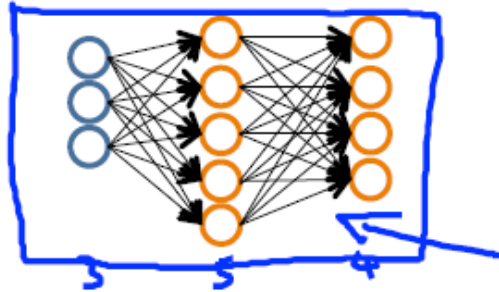
$y \in \mathbb{R}^K$  E.g.  $\begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}$   
pedestrian car motorcycle truck

K output units



## Training a neural network

Pick a network architecture (connectivity pattern between neurons)



→ No. of input units: Dimension of features  $x^{(i)}$

→ No. output units: Number of classes

Reasonable default: 1 hidden layer, or if >1 hidden layer, have same no. of hidden units in every layer (usually the more the better)

$$y \in \{1, 2, 3, \dots, 10\}$$

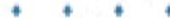
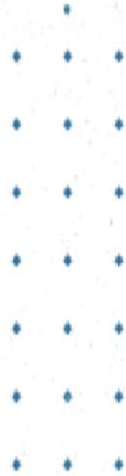
$$y = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \\ \vdots \\ 0 \end{bmatrix} \text{ or } \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \\ \vdots \\ 0 \end{bmatrix} \text{ or } \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \\ \vdots \\ 0 \end{bmatrix}$$



# Training a neural net

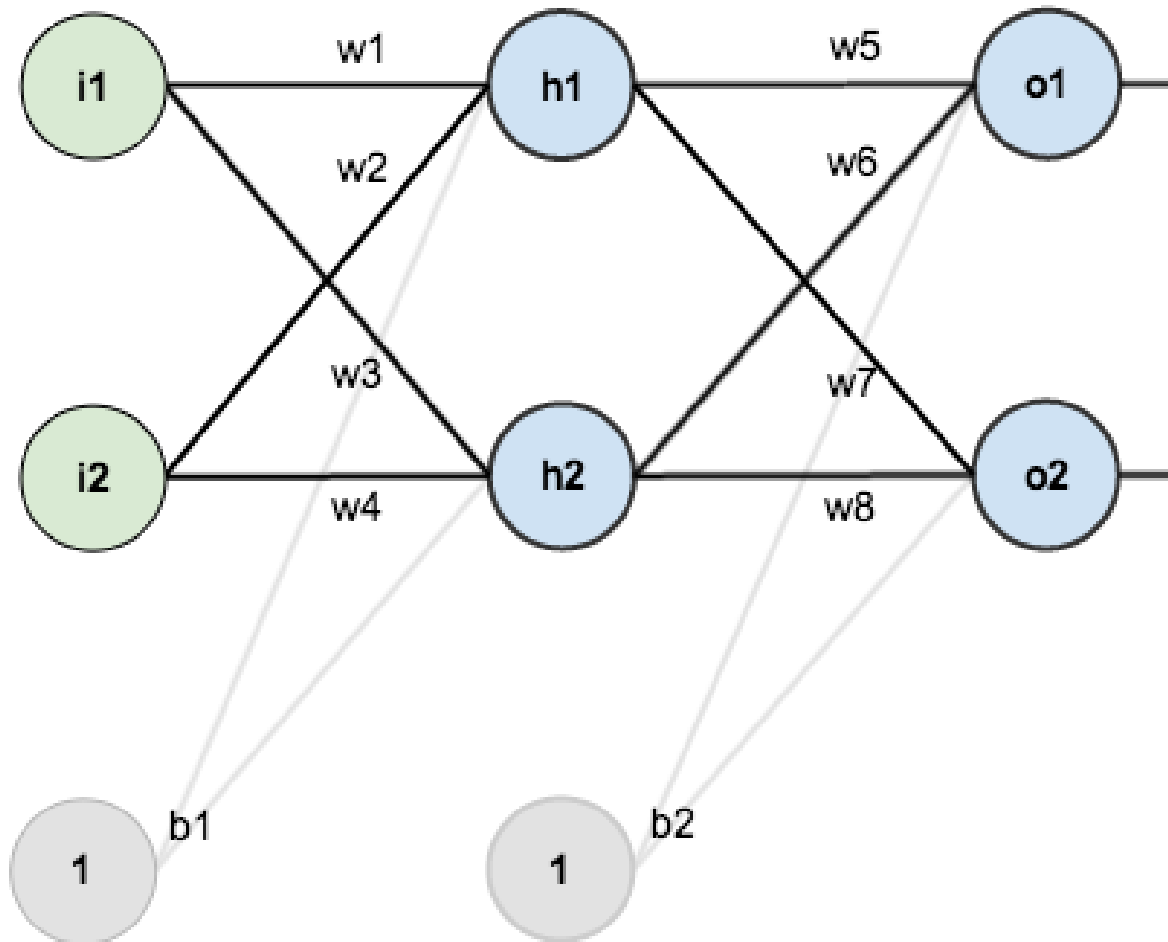
1. Randomly initialize weights
2. Implement forward propagation to get the output at each neuron
3. Compute the error at the output layer  $E_{total}$
4. Implement backpropagation to compute partial derivatives  $\frac{\partial E_{total}}{\partial w_{jk}^l}$
5. Use Gradient descent or any other optimization technique to update the weights to minimize  $E_{total}$
6. Repeat this process over multiple iterations (epochs) until the error converges

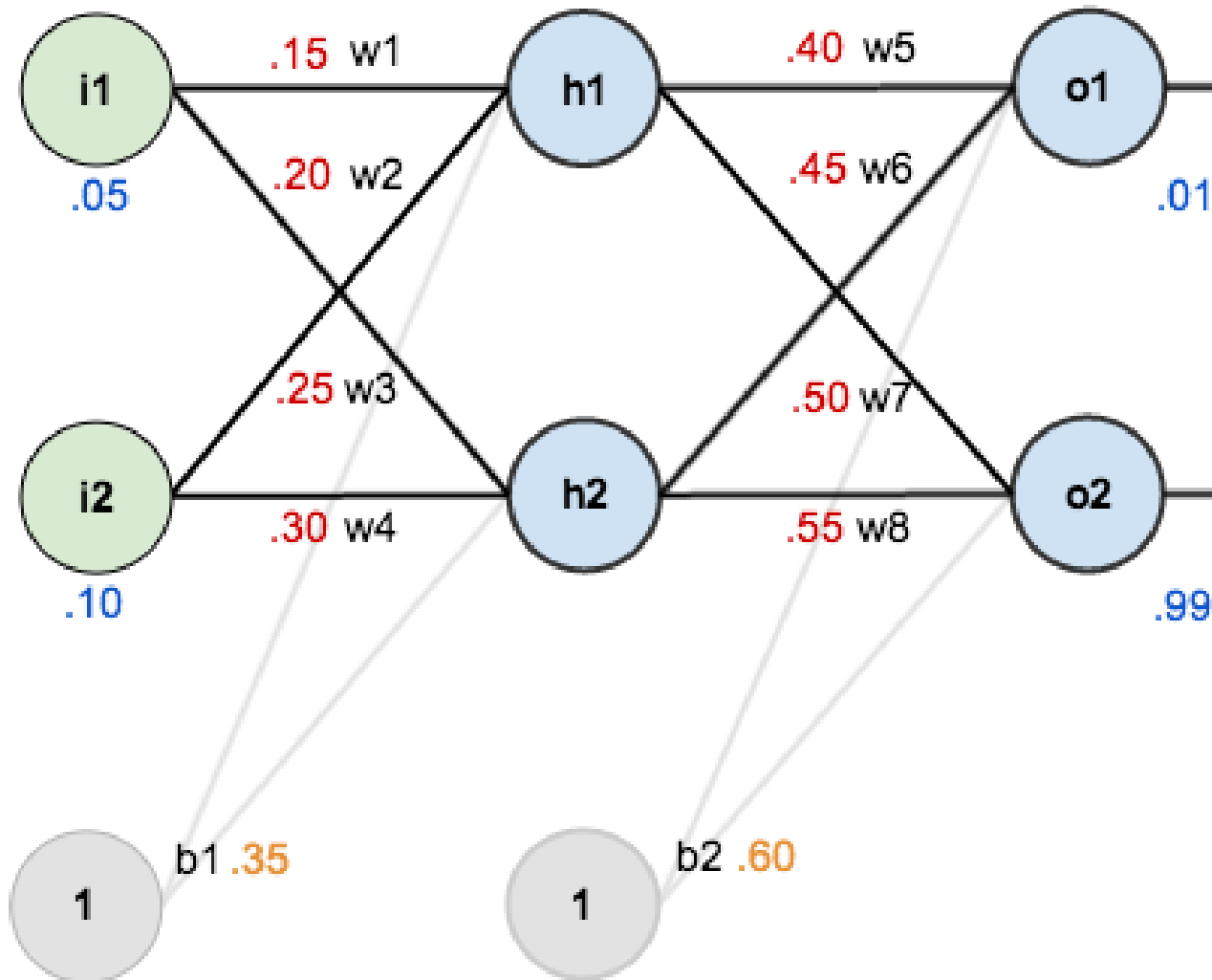
# Chain rule



# Step 4 - Backpropagation



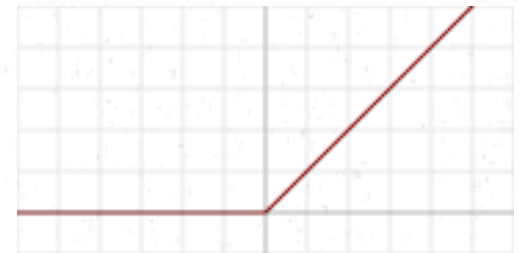






# Activation functions

- Identity
- Step
- Sigmoid
- ReLU
- TanH



# Additional Reading

- [book13.dvi \(stanford.edu\)](#)
- [What is backpropagation really doing?](#)
- [Backpropagation calculus](#)